Checkpoint evaluation: from ladder loss to usable-model evidence

Fixed holdouts, task protocols, exports, recurrence parity and automatic records

Evaluation is part of the trainer qualification

Build the evaluator before large ladder runs. It must answer three questions: is the training computation correct; which recipe improves the fixed target distribution; and does the model acquire the capabilities we care about? Training loss alone cannot answer all three, especially when data mixtures change.

The TorchTitan validation guide separates in-loop validation from external task evaluation after checkpoint conversion. Megatron likewise has in-loop validation and distributed checkpointing; a training checkpoint is not automatically a Hugging Face model. The LM Evaluation Harness is downloaded at commit `d6de81643928d653435c431bae19945d41d32520`; its tracked source is archived in `references/evaluation/2026-lm-evaluation-harness/`.

Freeze a three-level suite

Level	Proposed use	Measurements and controls
Numerical fixtures	Every trainer/export/loop change	Logits, masked NLL, gradients, optimizer step and full-forward/cache parity on known token IDs
Fixed distribution losses	Every selected checkpoint in every ladder run	Held-out web quality bands, code, math, scientific text, cyber domain; token-weighted NLL plus BPB where useful
Downstream capability tasks	Milestones and all final annealed endpoints	Fixed task versions, few-shot settings, prompts, extraction rules, accuracy/normalized accuracy/exact match and uncertainty

Proposed cheap task slice: HellaSwag, PIQA, ARC-Easy/Challenge and WinoGrande. Proposed fuller milestones: MMLU, GSM8K and a carefully versioned code-generation benchmark, plus internally held-out cyber tasks. These are candidate evaluation choices, not a claim that every task has statistical power at tiny proxy sizes. Use published task configurations from the pinned harness; audit dataset versions and access before freezing names.

Tiny models often score near chance. Keep continuous choice log-likelihood/margin or domain BPB available, and do not select a mixture solely on chance-level task accuracy. AI2's task-scaling report in our library explicitly motivates separating language-model/task loss from bounded accuracy prediction.

Define loss precisely

For fixed held-out token positions $\mathcal{V}$, model conditional probabilities p_θ, and binary loss mask m_i,

$$\mathrm{NLL}_{\mathrm{tok}} = -\frac{\sum_{i\in\mathcal{V}} m_i \log p_\theta(x_i \mid x_{<i})}{\sum_{i\in\mathcal{V}} m_i}.$$

Exclude padding and declared context-only positions. Record whether EOS is scored, whether documents share context and how long-document windows overlap. Average total loss sums and token counts across ranks; an unweighted average of microbatch means can bias results when token counts differ. Router regularizers are training diagnostics and should not be silently included in held-out next-token NLL.

For the same raw UTF-8 evaluation text,

$$\mathrm{BPB} = \frac{-\sum_i \log p_\theta(x_i \mid x_{<i})}{\ln 2 \; n_{\mathrm{bytes}}}.$$

State text normalization, byte accounting, document boundary conditioning and special-token scoring. BPB permits comparison across tokenizers only when raw text and scoring semantics are comparable. Do not call BPB “BPC”: Farseer uses a character denominator, whereas Marin’s records report bytes. They differ on Unicode text.

Freeze domain weights v_b before comparing mixtures: $L_{\mathrm{target}} = \sum_b v_b L_b$. Training runs may use different training weights w_b, but the evaluator’s v_b stays fixed. A lower own-mixture training loss may merely mean sampling easier text.

Protect model selection from leakage

Create three boundaries: training data; development evaluation used for recipe selection; and final held-out tasks/slices used only for acceptance. Remove benchmark-derived training documents before tokenization/export. Split duplicate and synthetic-parent families together. Check evaluation paths and checksums as carefully as training manifests.

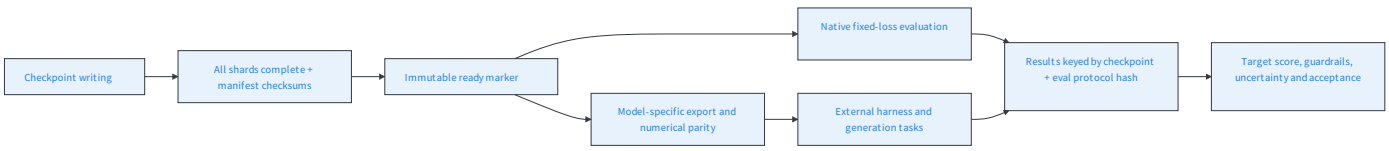
Use paired item differences for checkpoint/recipe comparisons and paired run seeds where possible. Bootstrap independent documents/items or run families, not correlated tokens from one paragraph. When many mixtures are screened, rerun finalists with fresh seeds and a held-out slice. Sibling cooldown branches sharing a prefix should remain in the same fold of surrogate cross-validation.

The Marin legacy records store `n_eval_tasks`, effective sample counts and several grouped BPB objectives. Different task coverage can move macro means even without a model improvement. Recompute scores from the same common task set before comparing rows; do not rank unlike macro averages blindly.

Checkpoint lifecycle and cadence

Our proposed default is lightweight fixed-loss validation every 1–2% of training tokens, cheap tasks at roughly 10% milestones, and the full suite at phase boundaries and fully annealed endpoints. Adjust frequency after measuring evaluation overhead. At tiny proxy scales, final-only fuller tasks may be more efficient. Checkpoint frequency for fault recovery is a separate wall-time decision.

Use an explicit lifecycle:



Retries must be idempotent: the same checkpoint/protocol hash maps to the same result key. Missing shards, a still-writing checkpoint or a failed export must produce failure status rather than an empty score. Store task-level outputs and enough metadata to rerun the measurement. Avoid loading a checkpoint while an asynchronous save is incomplete.

Native validation configuration

In the inspected TorchTitan version, configure `Validator.Config(freq=..., steps=..., dataloader=...)` in a Python recipe. Its default can use C4 validation; **replace that with our immutable holdouts**. The guide supports `repeat=False`, permitting complete finite evaluation. Training and validation should have separate sampler state.

In Megatron, the example uses `--eval-interval` and `--eval-iters`; these are updates and batches, not tokens or a declaration of a fixed full evaluation set. Convert to loss-bearing token coverage and ensure repeated evaluations select the intended identical holdout. Inspect the pinned training evaluator and token-weighted loss tests before relying on its aggregation for packed variable-length data.

If external evaluation borrows hero GPUs, count that time in the budget. A separate asynchronous worker avoids stalls but is additional compute, not free capacity.

Export and independent evaluator parity

For a **supported standard Llama3 8B**, TorchTitan's pinned converter exposes this illustrative interface:

```
cd /mnt/shared/repos/scaling-ladder-knowledge/tmp/torchtitan
python scripts/checkpoint_conversion/convert_to_hf.py \
  /checkpoints/standard-llama3/step-1000 \
  /eval-models/standard-llama3-step-1000 \
  --hf_assets_path /datasets/llama3-8b-assets \
  --model_name llama3 --model_flavor 8B --export_dtype bfloat16
```

These are placeholder paths; this command has not been executed. The converter's defaults and model registry are architecture-specific. Preserve the correct tokenizer and supply an architecture-matching `config.json`; weight serialization does not synthesize a correct arbitrary custom model definition.

For Megatron, use the selected release's compatible model-specific conversion tooling or a tested native harness adapter. The repository's GPT-OSS examples demonstrate a particular conversion route, not a universal converter for a new MoE. Obtain a known standard checkpoint, round-trip it and compare logits before exporting our own architecture.

TorchTitan's conversion README recommends greedy-decoding checks and a distributional/KL test; it illustrates how a missing RoPE permutation can produce subtle errors. Adopt both, with measured precision-dependent tolerances on fixed inputs. Also compare held-out NLL and token IDs. Passing a shape check or producing fluent output is insufficient.

For the proposed repeated middle blocks, create a custom model definition or native adapter that executes shared weights in the correct order and maintains **separate KV cache entries per visit**. Compare cached generation to full-prefix recomputation. Do not label its weights as a standard non-looped Qwen/Llama configuration. Until this path passes, external scores are not trustworthy.

Harness invocation and task controls

In a separate evaluation environment built from the pinned harness, a **standard supported HF model** can be tested with:

```
lm_eval --model hf \
  --model_args pretrained=/eval-models/standard-model,dtype=bfloat16 \
  --tasks hellaswag --num_fewshot 0 --batch_size 8 \
  --output_path /eval-results/standard-model/hellaswag \
  --log_samples
```

Run 0-shot ARC/PIQA/WinoGrande as separately frozen tasks or a documented group; run 5-shot MMLU and the chosen GSM8K few-shot protocol separately. A single global `num_fewshot` should not accidentally overwrite different intended protocols. Pin task YAMLs, dataset revisions, harness seed, tokenizer, maximum context, generation stop strings, temperature and answer parser. Base-model evaluation should not silently add an instruction chat template.

A vLLM backend may accelerate supported models, but custom recurrent MoE support requires separate model/cache integration. Use a native scorer or qualified HF backend first. Tool/code execution benchmarks need a controlled execution environment; their result is not interchangeable with multiple-choice code knowledge.

Result record and acceptance

Every result should contain the run/config hash, corpus release, checkpoint token count, stored/active/executed parameter counts, training seed, phase and decay status, evaluator commit, task/data revision, token/byte counts, per-domain NLL/BPB, task metrics with uncertainty, throughput and errors. Keep full task metrics alongside the aggregate.

A proposed acceptance gate is: no fixture/export/cache failure; no unresolved contamination; a positive paired improvement on the predeclared target objective; and no material regression on critical capability/domain guardrails beyond measured noise. Set the actual tolerance before seeing finalists. Compare fully annealed endpoints at matched exposure and also wall-clock quality, since a slower recurrent model may lose a nominal FLOP advantage.

No checkpoint evaluation has been run here. This guide defines the integration and evidence needed to evaluate the future ladder and hero run consistently.